

MIS410 — Week 7 Debrief

Database and SQL | Experimental Analysis, Statistics & Python Integration

Week 7 Overview

This week combined a deep dive into inferential statistics (chi-square, t-tests, z-tests, ANOVA, natural experiments, and regression discontinuity design) with a full applied project: an A/B testing discussion post using a fitness industry example, two classmate replies, and a four-part critical thinking assignment combining SQL and Python for experimental analysis on a customer reviews dataset.

1. Core Statistical Concepts

Chi-Square: Three Types

Chi-square answers: *"Are these differences in counts real, or just random noise?"* Three flavors:

- **Goodness of Fit** — one variable; checks actual counts against an expected/predicted split (e.g., "expected 120 signups, got 135 — is that gap meaningful?").
- **Test of Independence** — two variables, one population; checks if two categorical variables are related (e.g., does variant assignment relate to onboarding completion?).
- **Test of Homogeneity** — one variable, separate populations; checks if distinct groups look the same on that variable (e.g., do two states have the same voting preference distribution?).

Chi-Square Walkthrough (Goodness of Fit example)

Gym example: expected 120 of 200 new members to opt in with a trainer; actual was 135.

	Expected	Actual
Trainer	120	135
No Trainer	80	65

Steps: (1) Find the gap in each cell → 15 in each. (2) Square the gap and divide by expected: $(15^2 \div 120) = 1.875$ and $(15^2 \div 80) = 2.8125$. (3) Sum → chi-square = 4.6875. (4) Degrees of freedom = categories – 1 = 1. (5) Compare to the threshold at $df=1$, $\alpha=0.05$ → 3.84. Since $4.6875 > 3.84$, the gap is statistically significant — real demand exists for the trainer option beyond initial projections.

T-Test vs. Z-Test

Both compare **averages** (not counts) between groups, asking: "is the gap between these averages real, or random noise?" Z-test requires knowing the true population spread in advance (rare in practice); t-test estimates spread from the sample itself and is the far more common real-world tool. Rule of thumb: small sample and/or unknown population spread → t-test.

Worked example — gym retention: Group A (no trainer) averages 375 days retained; Group B (trainer) averages 500 days, $sd=150$ both groups, $n=50$ each. Gap = 125 days. Standard error =

$\sqrt{[(150^2/50)+(150^2/50)]} = \sqrt{900} = 30$. $t = 125 \div 30 = 4.17$. $df = 50+50-2 = 98$. Threshold at $df=98$, $\alpha=0.05 \approx 1.98$. Since $4.17 > 1.98$, the retention gap is statistically significant — strong evidence trainer access genuinely extends retention.

Comparison Table: Chi-Square vs. T-Test vs. Z-Test vs. ANOVA

Test	Compares	Score Formula Shape
Chi-square	Counts in categories	$(\text{actual}-\text{expected})^2 \div \text{expected}$, summed
T-test	2 averages	$\text{gap} \div \text{standard error}$
Z-test	2 averages (known spread)	$\text{gap} \div \text{standard error (known spread)}$
ANOVA	3+ averages	$\text{between-group variation} \div \text{within-group variation}$

ANOVA

Extends the t-test to 3+ groups at once, avoiding the inflated false-positive risk of running many pairwise t-tests. Core question: "Is the difference *between* group averages bigger than the natural wobble *within* each group?" Produces an F-statistic; a larger F suggests a genuine difference exists somewhere among the groups — but ANOVA alone doesn't say *which* group differs (a follow-up post-hoc test would).

Experiment Eligibility & Dilution Issues

- **Too many cohorted (dilution):** irrelevant/ineligible users get swept into the experiment (e.g., a new-user-only test accidentally includes everyone). Fix: add a JOIN/WHERE filter to restrict to the truly eligible population, then re-check sample size afterward.
- **Too few cohorted:** check (a) is the remaining sample big enough for significance, and (b) is there a systematic pattern in who got excluded (e.g., only old phone models) — non-random exclusion can bias results toward or against certain demographics.
- **Broken randomization:** the "coin flip" assigning variants isn't truly random — e.g., the same engaged user lands in both groups, or highly active users cluster in one variant. "Too good to be true" results are a red flag; check for duplicate assignment and uneven pre-experiment engagement across groups.

Natural Experiment Analysis

A natural experiment uses an accidental, uncontrolled split (time period, policy change, geography) instead of true random assignment. Example from class: comparing email opt-in rates "pre" vs. "post" a date cutoff. Risk: other factors could have changed at the same time as the split, so causal claims must be more cautious than with a true randomized test ("associated with" rather than "caused by").

Regression Discontinuity Design (RDD)

Uses a sharp cutoff (a "running variable" crossing a threshold) to compare units just above vs. just below the line — since landing on either side is nearly arbitrary right at the cutoff, this approximates a true random comparison. **Sharp RDD:** the cutoff is a hard rule (score 90+ = award, always). **Fuzzy RDD:** crossing the cutoff only makes treatment more *likely*, not guaranteed. Validity checks: (1) confirm people can't manipulate

which side they land on (watch for suspicious pile-ups near the cutoff), (2) confirm groups look similar on other traits except the treatment, (3) confirm the outcome shows a genuine jump at the cutoff rather than a gradual drift.

2. SQL Query Concepts Practiced

Worked through several real SQL queries this week, reinforcing JOIN mechanics, filtering, and aggregation:

- **Onboarding completion query:** used a LEFT JOIN with the join condition (`b.action = 'onboarding_complete'`) placed in the **ON** clause rather than WHERE, so non-matches remain as rows instead of vanishing — essential for the `COUNT(CASE WHEN . . .)` pattern to correctly split matched vs. unmatched users into separate columns in one pass.
- **Pre/post email opt-in query:** a CASE WHEN split users into two time-based cohorts (a natural experiment, not true randomization) — reinforced that before/after comparisons need more cautious interpretation than randomized tests.
- **JOIN syntax pattern:** repeatedly practiced the `table.column = table.column` shape for ON conditions, and the GROUP BY rule that you can only group by columns that exist *before* aggregation (not by a calculated alias like `average_rating`).

Key Debugging Lesson: Connection ≠ Data

A significant real-world lesson surfaced during Task 2's Python/SQLAlchemy setup: **having a working database connection does not mean the data is actually there**. The debugging chain, in order:

- Database name mismatch — connection string pointed to a database (MISMOD7) that either didn't exist or wasn't the right one; renamed to `mis410_experiment`.
- Database existed but was empty — no `customer_reviews` or `products` tables inside it yet. A database and its tables are two different levels — like a filing cabinet vs. the folders inside it.
- Fix required loading the provided CSV files with `pd.read_csv()`, then pushing them into MySQL as real tables with `.to_sql()`.
- File path errors along the way: a placeholder username left in the path, then a backslash-vs-forward-slash issue (Windows paths use `\`, but Python treats backslash as an escape character in strings — fixed by using forward slashes).

Takeaway: this is a common real-world environment-setup gotcha, not just a coursework quirk — worth remembering for future database work.

3. Critical Thinking Assignment — Full Build

Dataset: fictional customer reviews (review_id, product_id, customer_id, rating, review_text, review_date) joined to products (product_id, category, price, launch_date).

Task 1 — SQL Queries

1. 2024 reviews only (review_id, product_id, rating, review_date):

```
SELECT review_id, product_id, rating, review_date
FROM customer_reviews
WHERE review_date BETWEEN '2024-01-01' AND '2024-12-31'
```

2. Reviews rated 4 or 5:

```
SELECT review_id, product_id, rating, review_text
FROM customer_reviews
WHERE rating = 4 OR rating = 5
```

3. Average rating by category (JOIN + GROUP BY):

```
SELECT category, AVG(rating) AS average_rating
FROM customer_reviews JOIN products
ON customer_reviews.product_id = products.product_id
GROUP BY category
```

4. Review count per product:

```
SELECT product_id, COUNT(review_id) AS review_count
FROM customer_reviews
GROUP BY product_id
```

Task 2 — Python Analysis

Data loading/cleaning: connected via SQLAlchemy, loaded the joined query into a DataFrame, converted review_date to a real datetime type with `pd.to_datetime()`, confirmed no nulls.

Descriptive statistics: mean, median, and standard deviation calculated for rating; distribution visualized with a histogram.

Hypothesis test (ANOVA): "There is a significant difference in the average ratings of products across different categories" — tested across all 8 categories (Electronics, Sports, Books, Home Appliances, Toys, Home Office, Beauty, Clothing).

Result: $F = 1.14$, $p = 0.33 \rightarrow$ since $p > 0.05$, fail to reject the null hypothesis. **No statistically significant difference in ratings between categories.**

Correlation analysis (price vs. rating): coefficient = $-0.05 \rightarrow$ negligible/no relationship. Visualized with a scatter plot (price on x-axis, rating on y-axis).

Task 3 — Combining SQL + Python

Time series: retrieved `product_id`, `review_date`, `rating`; extracted month from `review_date` (`.dt.month`), grouped and averaged rating by month. Result: ratings stayed narrow (2.75–3.21) across all 12 months, no strong seasonal pattern — months 6, 7, and 11 trended slightly higher, month 4 dipped lowest.

Sentiment analysis: retrieved `review_text` and `rating`; used NLTK's VADER (`SentimentIntensityAnalyzer`) via `.apply(lambda text: sia.polarity_scores(text)['compound'])` to score every review.

Result: correlation between sentiment score and rating ≈ -0.0003 (essentially zero). **Review tone does not predict star rating in this dataset** — a genuinely counterintuitive finding worth highlighting, since sentiment and rating would naturally be expected to align.

Visualizations completed: (1) bar plot — average rating by category, (2) time series line plot — average rating by month, (3) scatter plot — price vs. rating, plus a bonus scatter plot — sentiment vs. rating.

Task 4 — Interpretation & Business Insights (submitted)

Report covered: methodology (SQL retrieval + Python cleaning/analysis pipeline), implications (ANOVA, correlation, sentiment, and time-series results synthesized together), business applications (category-specific fixes unlikely to move satisfaction; price isn't a satisfaction lever; sentiment/rating disconnect flagged as worth investigating further), and challenges (new-concept syntax for ANOVA/F-statistics, and the MySQL database/table debugging saga).

4. Discussion Post & Peer Replies

Built a gym A/B testing example: Group A (free membership, no trainer obligation) vs. Group B (3-month trainer-inclusion obligation), hypothesizing Group B retains longer. Used the chi-square goodness-of-fit test (135 actual vs. 120 expected trainer signups, $\chi^2=4.6875 > 3.84$ threshold) to show real signup demand for the trainer option, and proposed a matplotlib bar chart (expected vs. actual, annotated with the chi-square score) to communicate results to non-technical stakeholders.

Flagged as a potential Data Analytics portfolio piece — the chi-square signup analysis plus the separately-built t-test retention concept (375 vs. 500 days, $t \approx 4.17$) could be adapted into a standalone project with a synthetic dataset, SQL cohorting, Python stats, and visualization.

Replies: responded to a classmate's Boeing supplier example, asking an open question about which variables would need to be controlled for a true A/B comparison and whether "defect rates" should break down by defect type/severity rather than treating all defects equally. Also replied to a classmate's Target store-layout example, introducing the concept of a heatmap overlay on a floor plan to visualize where foot traffic/purchases shifted after a redesign.

5. Ideal Conditions for A/B Testing (discussion answer)

- True random assignment — no factor sways who lands in which group.
- Large enough sample size to detect a real effect against natural noise.
- Only the intended/eligible population included (no dilution).
- One variable changes at a time, so effects can be isolated.
- Consistent time period, free of external interference (holidays, outages, other campaigns).
- Independence between users (one person's outcome shouldn't influence another's).
- A clear, predefined success metric decided before seeing results.

Reflection

Strong week for connecting theory to practice — every statistical concept (chi-square, t-test, ANOVA, RDD, natural experiments) was immediately grounded in a concrete SQL/business example rather than staying abstract. The MySQL debugging saga, while frustrating in the moment, reinforced a genuinely important real-world lesson about the difference between a working connection and actual data availability. Also demonstrated growing instinct for catching conceptual mismatches independently — correctly flagging that ANOVA didn't fit a 2-group scenario before being told, and pushing back precisely on a peer's semantic-vs-substantive reframing of a logos argument.